



万字长文！大模型(LLM)推理优化技术总结（非常详细）



NLP 自然语言处理
微信公众号：AINLPer

📖 收录于 · 大模型基础知识 >

285 人赞同了该文章 >

首发：AINLPer 微信公众号（每日论文干货分享！！）

编辑：ShuYini

校稿：ShuYini

时间：2025-6-17

更多精彩内容--> [专注大模型/AIGC、Agent、RAG等学术前沿分享！](#)

引言

大模型训练成本很高，且在推理过程中需要大量的计算资源，为了能够实现大模型应用落地，需解决大模型推理成本、模型响应速度等问题，这就需要对大模型进行推理优化。为此，本文将详细介绍主流的大模型推理优化技术，文章安排如下：





本文相关内容需要大家对 [Transformer架构](#) 和注意力机制有一个基本的了解。不了解的小伙伴可以参考以下文章：

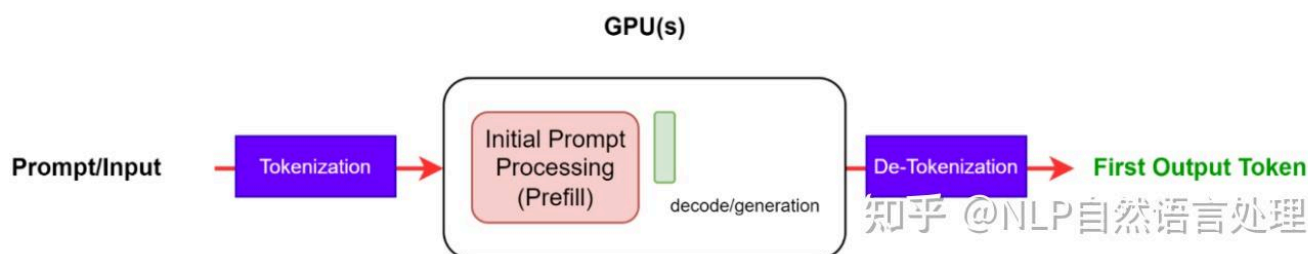
[Transformer | 一文了解：缩放、批量、多头、掩码、交叉注意力机制 \(Attention\)](#)

[2万字长文！一文了解 Attention，从MHA到DeepSeek MLA，大量图解，非常详细！](#)

1. 什么是 LLM 推理

大多数流行的 only-decode LLM（例如 [GPT-4+](#)、[Qwen 系列](#)）都是针对因果建模目标进行预训练的，本质上是作为下一个词预测器。这些 LLM 将一系列 tokens 作为输入，并自回归生成后续 tokens，直到满足停止条件（例如，生成 tokens 数量的限制或遇到停止词）或直到生成特殊的 `<end>` 标记生成结束的 tokens。该过程涉及两个阶段：预填充阶段和解码阶段。

请注意，tokens 是模型处理的语言的原子部分。一个 tokens 大约是四个英文字符。所有自然语言在输入模型之前都会转换为 tokens。下图是大模型推理过程。



1.1 预填充阶段 (Prefill)

在预填充阶段，也可以理解为输入阶段。LLM 处理输入 token 以计算中间状态（keys 和 value），用于生成“第一个”token。每个新的 token 都依赖于所有先前的 token，但由于输入的全部已知，因此在运算上，都是高度并行化矩阵运算，可以有效地使用 GPU。

1.2 解码阶段 (Decode)

在解码阶段，可以理解为输出阶段。LLM 一次自回归生成一个输出 token，直到满足停止条件。每个输出 tokens 都需要直到之前迭代的所有输出状态（keys 和 values）。这与预填充输入处理相比，就像矩阵向量运算未充分利用 GPU 计算能力。数据（weights, keys, values, activations）从内存传输到 GPU 的速度决定了延迟，而不是计算实际时间消耗。即，这是一个内存限制操作。

本文中的许多推理挑战和相应的解决方案都涉及此解码阶段的优化：高效的注意力模块、有效管理键和值等。

不同的 LLMs 可能使用不同的 tokenizers，因此比较它们之间的输出 tokens 可能并不简单。在比较推理吞吐量时，即使两个 LLMs 每秒输出的 tokens 相似，如果它们使用不同的 tokenizers，也可能不相等。这是因为相应的 tokens 可能代表不同数量的字符。

1.3 批处理 (Batching)

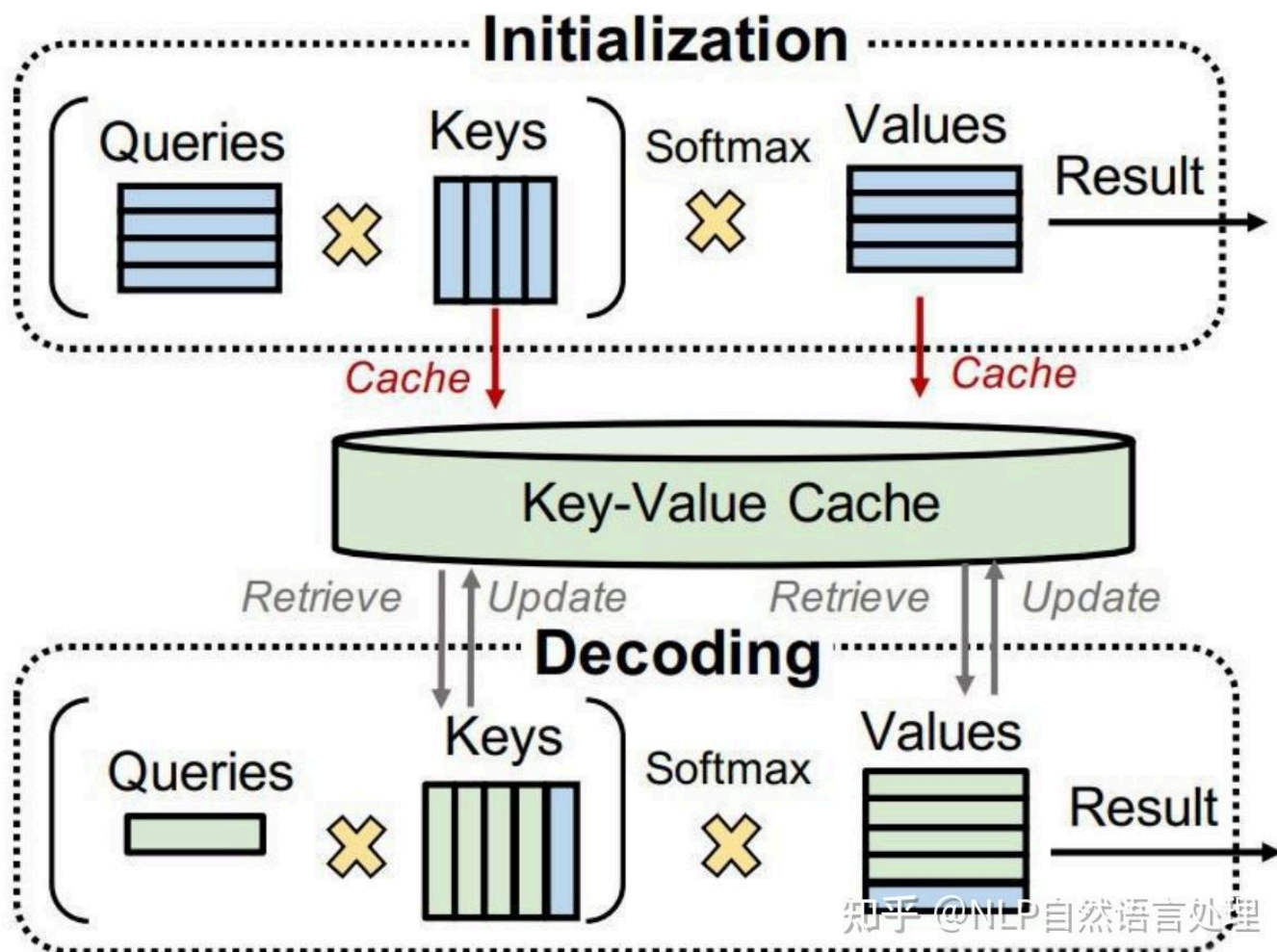
提高 GPU 利用率和有效吞吐量的最简单方法是通过批处理。由于多个请求使用相同的模型，因此权重的内存成本被分散。大批量数据传输到 GPU 一次处理，将提高 GPU 资源的利用率。然而，批量大小只能增加到一定限制，此时可能会导致内存溢出。为了防止这种情况发生，需要查看键值 (KV) 缓存和 LLM 内存要求。

传统批处理（也称为静态批处理，static batching）不是最佳的。这是因为对于批次中的每个请求，LLM 可能会生成不同数量的 tokens，并且不同 tokens 有不同的执行时间。因此，批次中的所有请求都必须等待最长 token 的处理完成，而生成长度的巨大差异可能会加剧这种情况。有一些方法可以缓解这种情况，例如稍动态批处理。

1.4 KV 缓存

解码阶段的一种常见优化是 KV 缓存。解码阶段在每个时间步生成单个 token，但每个 token 依赖于之前 token 的键和值张量（包括预填充时计算的输入 tokens 的 KV 张量，以及当前时间步之前计算的任何新 KV 张量）。

为了避免在每个时间步重新计算所有 tokens 的这些张量，可以将它们缓存在 GPU 内存中。每次迭代，当需要计算新 token 时，它们都会被添加到正在运行的缓存中，以便在下次迭代中使用。在一些实现中，模型的每一层都有一个 KV 缓存。下图展示了 KV 的缓存机制。



1.5 大模型内存需求

实际上，LLM对GPU显存的需求主要是模型权重和KV缓存：

- **模型权重**：模型参数占用内存。例如，具有 70 亿个参数的模型（例如 Llama 2-7B），以 16 位精度（FP16 或 BF16）加载，将占用大约 $7B * \text{sizeof}(\text{FP16}) \approx 14 \text{ GB}$ 的内存。
- **KV 缓存**：自注意力张量的缓存占用内存，避免冗余计算。

使用批处理时，批处理中每个请求的 KV 缓存仍然必须单独分配，并且可能会占用大量内存。下面的公式描述了 KV 缓存的大小，适用于当今最常见的 LLM 架构。

每个Token KV缓存 (字节) $= 2 * (\text{num_layers}) * (\text{num_heads} * \text{dim_head}) * \text{precision_in_bytes}$

第一个因子 2 代表 K 和 V 矩阵。通常， $(\text{num_heads} * \text{dim_head})$ 的值与 Transformer 的 `hidden_size`（或模型的维度，`d_model`）相同。这些模型属性通常可以在配置文件中找到。

输入批次中输入序列中的每个 tokens 都需要此内存大小。假设半精度，KV 缓存的总大小由以下公式给出：

总的Token KV缓存 (字节) $= (\text{batch_size}) * (\text{sequence_length}) * 2 * (\text{num_layers}) * (\text{hidden_size}) * \text{sizeof}(\text{FP16})$

例如，对于 16 位精度的 Llama 2-7B 模型，批量大小为 1，KV 缓存的大小将为 $1 * 4096 * 2 * 32 * 4096 * 2$ 字节，即约 2 GB。

可以发现 Llama 2-7B 本身模型权重占用为 14G，单个输入序列长度 4096 需要缓存为 2G，这就需要 16GB 的显存，要运行该模型可见单个显存的要求。为此如何高效的管理 KV 缓存就成为了一项重要的挑战。内存需求随着批量大小和序列长度线性增长，它限制了可服务的吞吐量，并对长上下文输入提出了挑战。这就是本文中介绍的多项优化背后的动机。

2. 模型并行

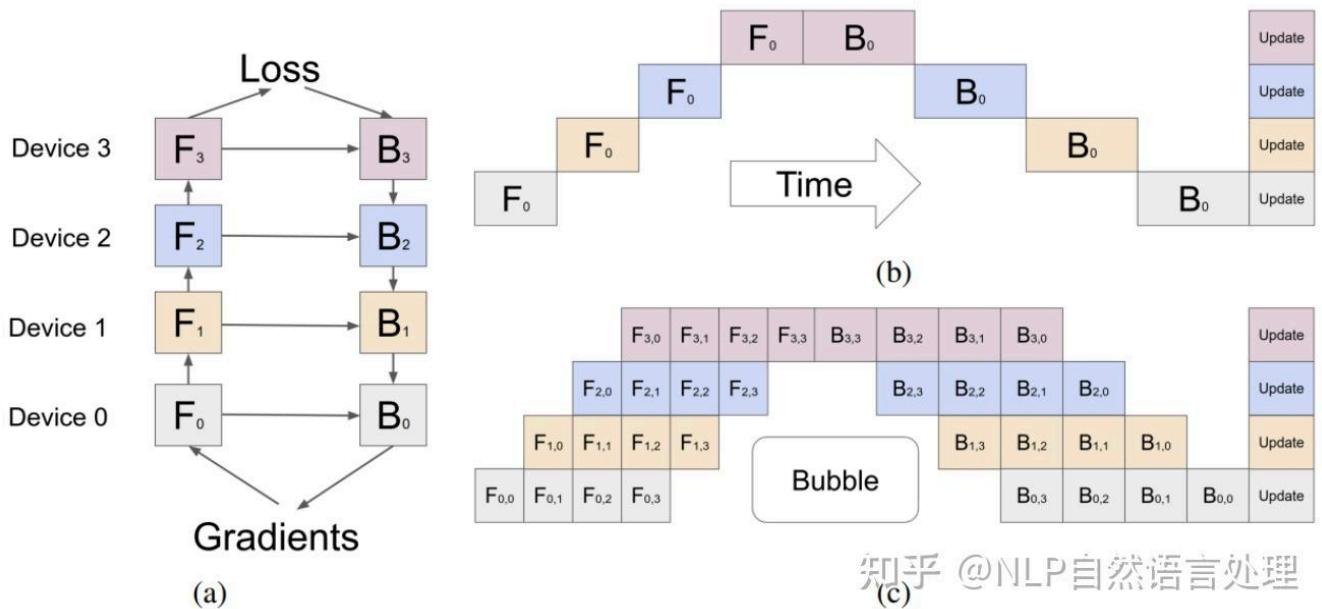
减少模型权重在每设备的显存占用的一种方法是将模型分布在多个 GPU 上。分散内存和计算可以运行更大的模型或更大批量的输入。模型并行化是训练或推理模型所必需的，模型并行化需要比单个设备更多的内存，用来训练和推理（延迟或者吐量）。根据模型权重的划分

方式，有多种方法可以并行化模型。

请注意，数据并行也是一种经常在与下面列出的其他技术相同的技术。在这种情况下，模型的权重被复制到多个设备上，并且输入的（全局）批量大小在每个设备上被分成微批次。它通过处理较大的批次来减少总体执行时间。然而，这是一种训练时间优化，在推理过程中不太相关。

2.1 Pipeline 并行⁺

Pipeline 并行化将模型（垂直）分片为块，其中每个块包含在单独设备上执行的层的子集。下图展示了四路 Pipeline，其中模型按顺序分区，并且所有层的四分之一子集在每个设备上执行。



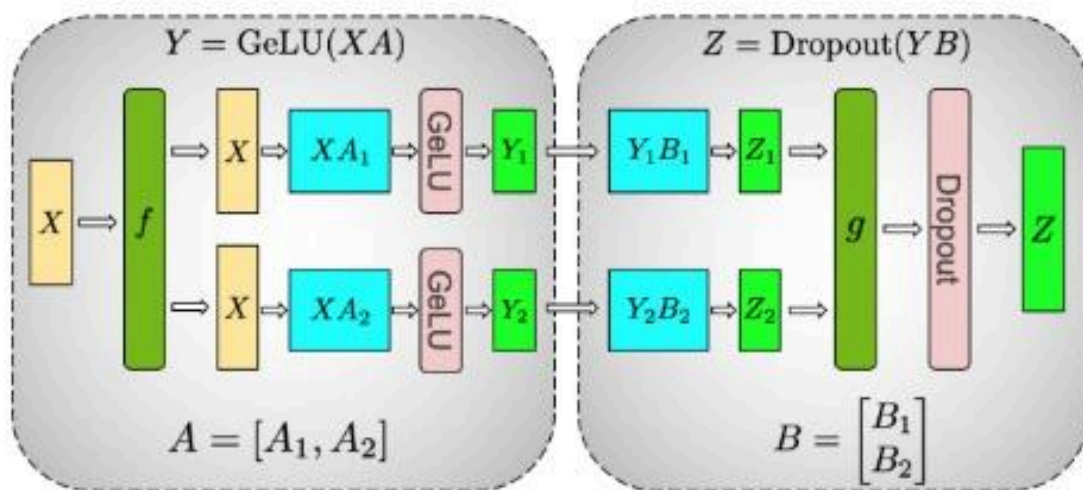
一个设备上的一组操作的输出被传递到下一个设备，后者继续执行后续块。 F_n 和 B_n 分别表示设备 n 上的前向传播和后向传播。每个设备上存储模型权重的内存需求被分成四份。

该方法的缺点是，由于处理的顺序性质，某些设备或层在等待前一层的输出（激活、梯度）时可能保持空闲状态。这会导致前向和后向传递效率低下或出现“Pipeline bubbles”。在上图（b）中，白色空白区域是 Pipeline 并行性产生的 Pipeline bubbles，其中设备闲置且未得到充分利用。

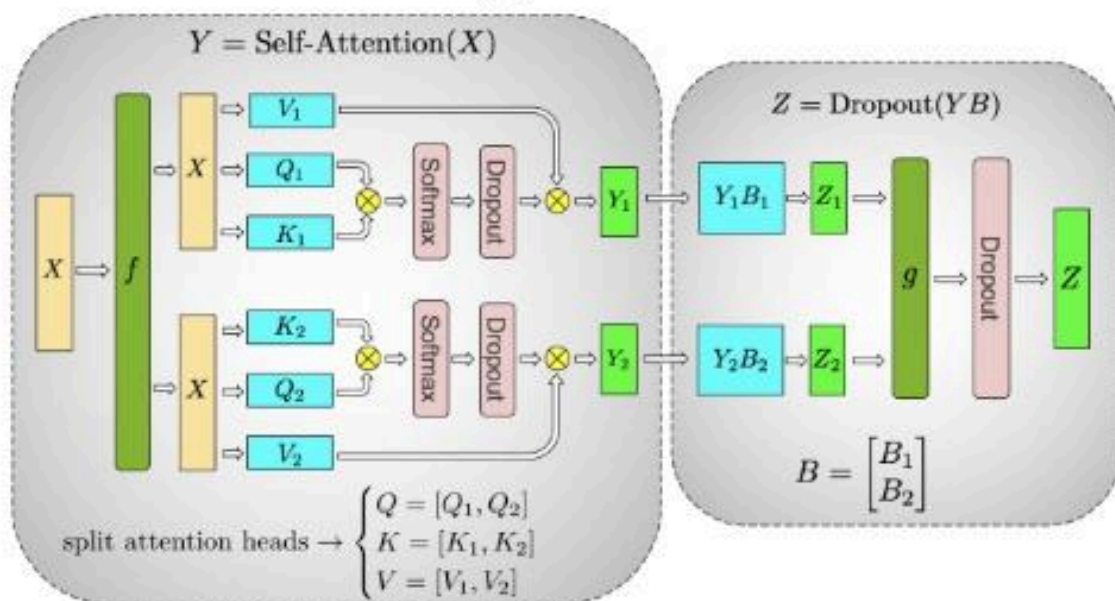
为了在一定时间内充分利用 GPU，可以通过微批处理的方式，即：分成多批，一个 GPU 完成之后立马安排下一次计算，但是这种方式可以在一定程度上缓解这种情况，如图 2c 所示。输入的全局批次大小被分成了批次，这些子批次被一一处理，最后累积梯度。请注意， $F_{n,m}$ 和 $B_{n,m}$ 分别表示设备 n 上 m 批次的前向和后向传递。这种方法缩小了管道气泡的尺寸，但并没有完全消除它们。

2.2 Tensor 并行⁺

Tensor 并行化将模型的各个层（水平）分片为更小的、独立的计算块，这些计算块可以在不同的设备上执行。Transformer 的主要组成部分，注意力块和多层感知器（MLP）层是可以利用 Tensor 并行化的。在多头注意力块中，每个头或一组头可以分配给不同的设备，以便它们可以独立且并行地计算。



(a) MLP



(b) Self-Attention 知乎 @NLP自然语言处理

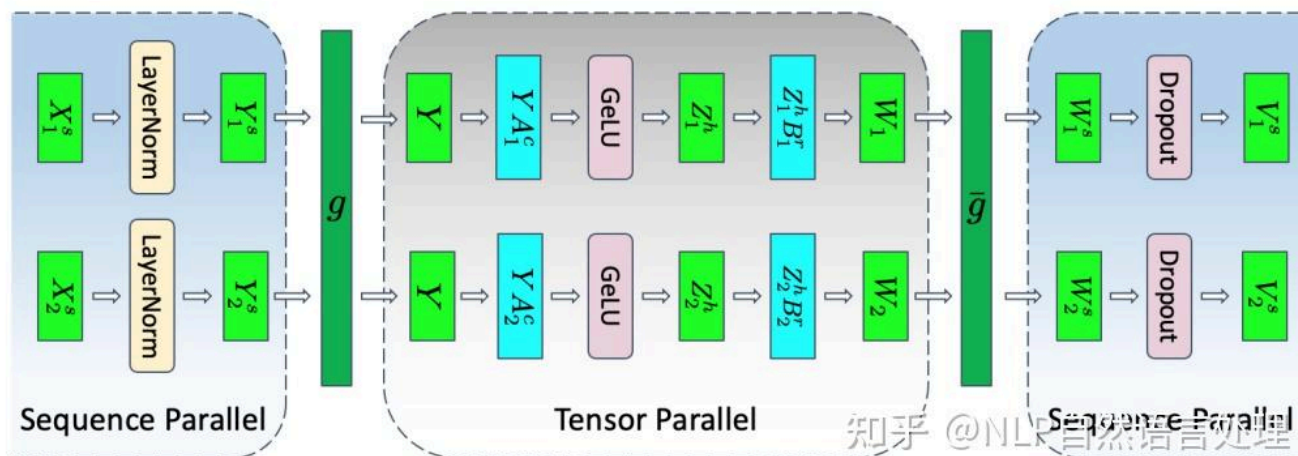
上图 (a) 显示了两层 MLP Tensor 并行的示例，每一层都由一个圆角框表示。在第一层中，权重矩阵 A 分为 A_1 和 A_2 。对于输入 X ，可以在同一批次不同设备上计算 XA_1 和 XA_2 ，其中， f 是 identity 操作。这将每个设备上存储权重的内存需求减半。归约操作 g 组合了第二层的输出。

上图 (b) 是自注意力层中 Tensor 并行的示例。多个注意力头本质上是并行的，并且可以跨设备分割。

2.3 Sequence 并行

Tensor 并行化是有局限性，它需要将层划分为独立的、可管理的块，不适用于 LayerNorm 和 Dropout 等操作，而是在 tensor 并行中复制。虽然 LayerNorm 和 Dropout 的计算成本较低，但它们确实需要大量内存来存储（冗余）激活。

如 Reducing Activation Recomputation in Large Transformer Models 所示，这些操作在输入序列中是独立的，并且这些操作可以沿着“序列维度”进行分区，从而提高内存效率。这称为序列并行性。



模型并行技术不是唯一的，可以结合使用。它们可以帮助扩展和减少 LLM 的每 GPU 内存占用量，但也有专门针对注意力模块的优化技术。

3. 注意力机制优化

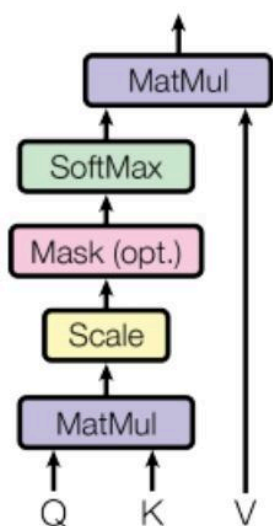
缩放点积注意力 (SDPA, scaled dot-product attention) 操作将 query 和 key 对映射到输出，如论文 Attention Is All You Need 所述。

3.1 多头注意力 (MHA)

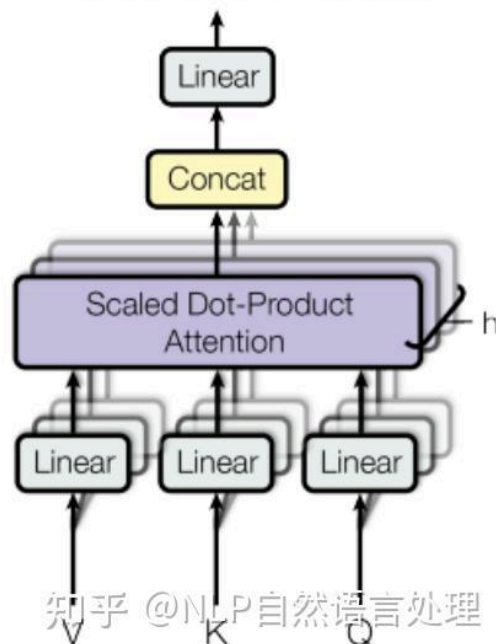
作为 SDPA 的增强，三个变换张量对 Q, K, V 分别进行线性变换，这些变换不会改变原有张量的尺寸，使模型能够共同关注来自不同位置的不同表示子空间的信息。这些子空间是独立学习的，使模型能够更丰富地理解输入中的不同位置。

如下图所示（缩放点积注意力（左）和多头注意力（右）），多个并行注意力操作的输出被拼接后线性投影以组合起来。每个并行注意力层称为“头”，这种方法称为多头注意力 (MHA)。

Scaled Dot-Product Attention



Multi-Head Attention



当使用八个并行注意力头时，每个注意力头的维度都会减少（例如： $d_{model}/8$ ）。这使得计算成本与单头注意力相似。

3.2 多查询注意力 (MQA)

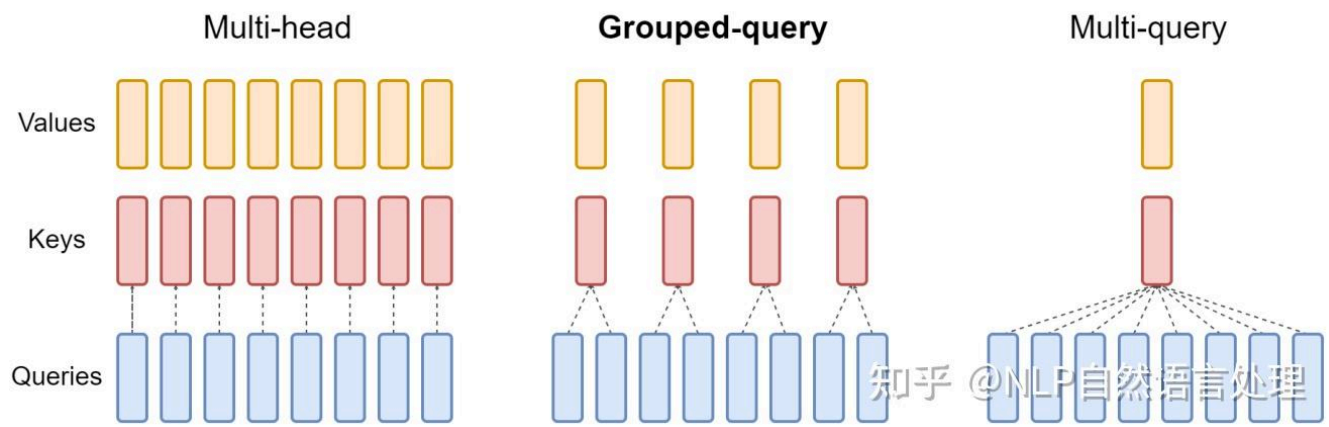
MHA 的推理优化之一称为多查询注意力 (MQA)，如 Fast Transformer Decoding 中提出的，在多个注意力头之间共享键和值。与以前一样，查询向量仍然被投影多次。

虽然 MQA 中完成的计算量与 MHA 相同，但从内存读取的数据量（键、值）只是以前的一小部分。当受内存带宽限制时，这可以实现更好的计算利用率。它还减少了内存中 KV 缓存的大小，为更大的批量大小留出了空间。

key 头的减少会带来潜在的准确性下降。此外，需要在推理时利用这种优化的模型需要在启用 MQA 的情况下进行训练（或至少使用大约 5% 的训练量进行微调）。

3.3 分组注意力（GQA）

分组查询注意力 (GQA) 通过将键和值投影到几组查询头，在 MHA 和 MQA 之间取得平衡（如下图所示）。在每个组中，它的行为类似于多查询注意力。



上图显示多头注意力有多个键值头（左）。分组查询注意力（中心）的键值头多于一个，但少于查询头的数量，这是内存需求和模型质量之间的平衡。多查询注意力（右）具有单个键值头，有助于节省内存。

最初使用 MHA 训练的模型可以使用原始训练计算的一小部分通过 GQA 进行“升级训练”。它们获得接近 MHA 的质量，同时保持接近 MQA 的计算效率。Llama 2 70B 是利用 GQA 的模型示例。

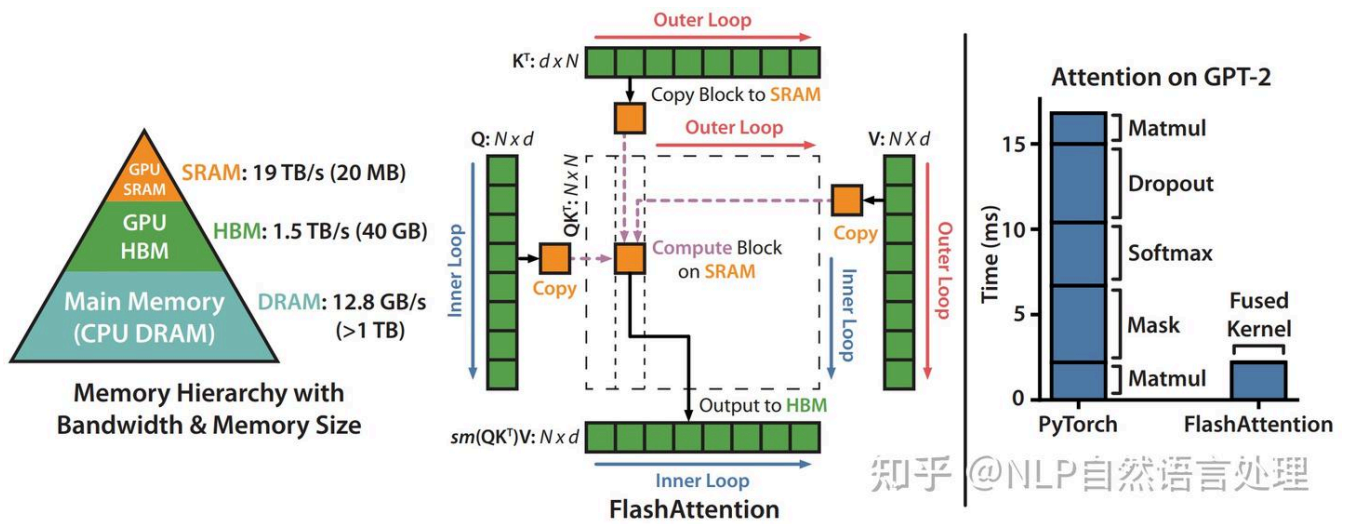
MQA 和 GQA 等优化通过减少存储的 key 头和 value 头的数量来帮助减少 KV 缓存所需的内存。KV 缓存的管理方式可能仍然效率低下。与优化注意力模块本身不同，下一节将介绍一种更高效的 KV 缓存管理技术。

3.4 Flash attention

优化注意力机制的另一种方法是修改某些计算的顺序，以更好地利用 GPU 的内存层次结构。神经网络通常用层来描述，大多数实现也以这种方式布局，每次按顺序对输入数据进行一种计算。这并不总是能带来最佳性能，因为对已经进入内存层次结构的更高、性能更高级别的值进行更多计算可能是有益的。

在实际计算过程中将多个层融合在一起可以最大限度地减少 GPU 需要读取和写入内存的次数，并将需要相同数据的计算分组在一起，即使它们是神经网络中不同层的一部分。

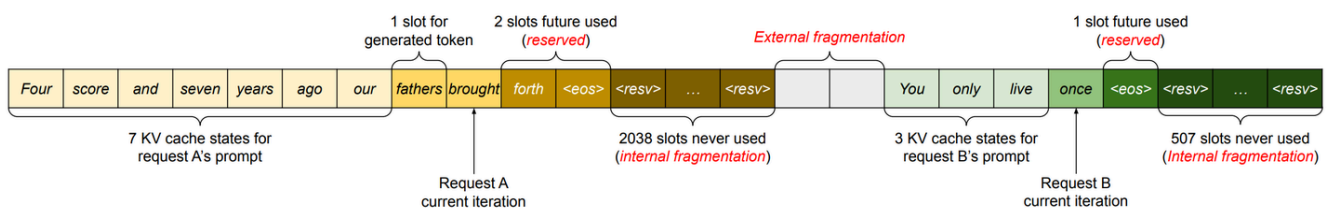
一种非常流行的融合是 **FlashAttention⁺**，这是一种 I/O 感知精确注意算法，详细信息请参阅 [arxiv.org/abs/2205.1413...](https://arxiv.org/abs/2205.14133)。精确注意力意味着它在数学上与标准多头注意力相同（具有可用于多查询和分组查询注意力的变体），因此可以无需修改即可交换到现有的模型架构，甚至是已经训练的模型。



I/O 感知意味着在将操作融合在一起时，它会考虑前面讨论的一些内存移动成本。特别是，FlashAttention 使用“平铺”一次性完全计算并写出最终矩阵的一小部分，而不是分步对整个矩阵进行部分计算，写出中间值。上图显示了 40 GB GPU 上的平铺 FlashAttention 计算模式和内存层次结构。右图显示了对注意力机制的不同组件进行融合和重新排序所带来的相对加速。

3.5 PageAttention

有时，KV 缓存会静态地“过度配置”(over-provisioned)，以考虑最大可能的输入（支持的序列长度），因为输入的大小是不可预测的。例如，如果模型支持的最大序列长度为 2,048，则无论请求中输入和生成的输出的大小如何，都将在内存中保留大小为 2,048 的数据。该空间可以是连续分配的，并且通常其中大部分未被使用，从而导致内存浪费或碎片。该保留空间在请求的生命周期内被占用。下图展示了由于过度配置和低效的 KV 缓存管理而导致的内存浪费和碎片。



为此，UC 伯克利的研究人员受操作系统分页的启发，提出了 PagedAttention(arxiv.org/pdf/2309.0618...)算法，PagedAttention 算法能够将连续的键和值存储在内存中的不连续空间中。它将每个请求的 KV 缓存划分为代表固定数量 tokens 的块，这些块可以不连续存储。

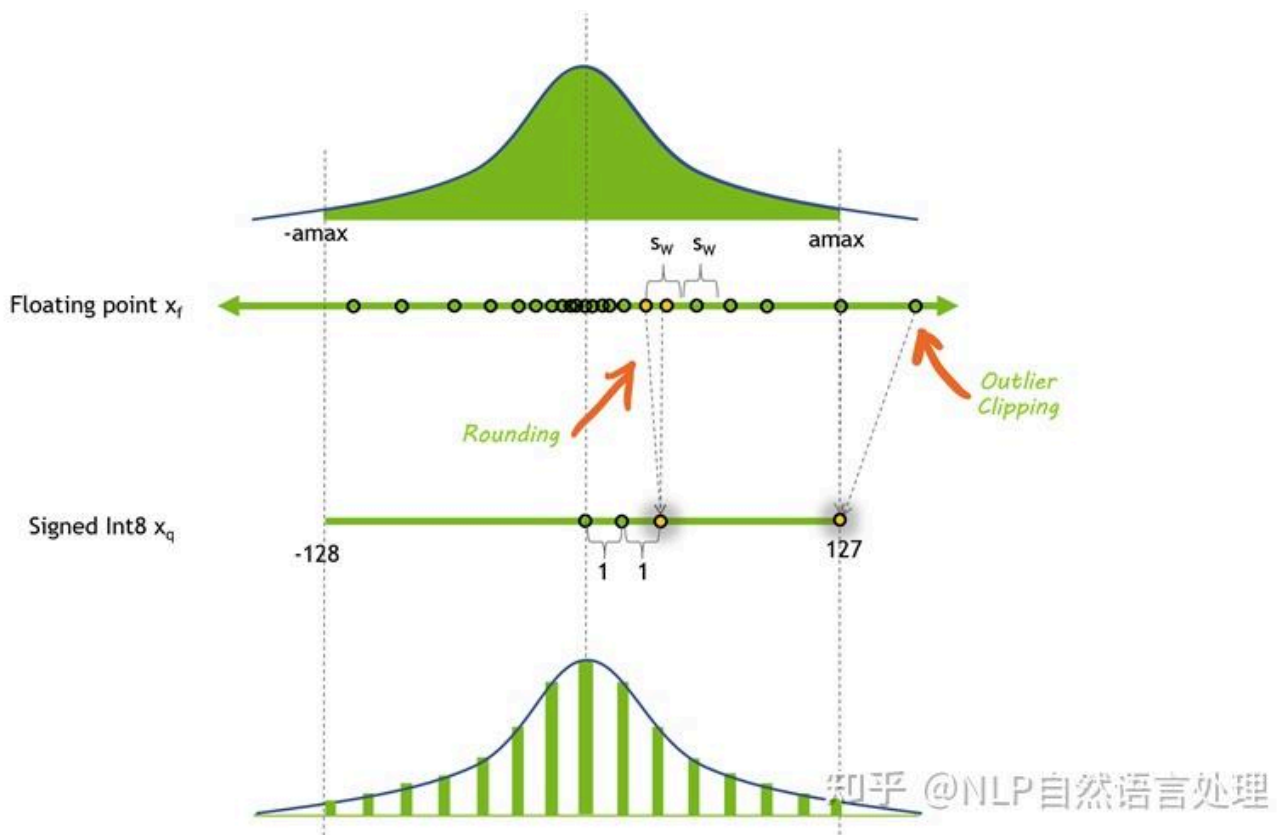
在注意力计算期间，使用根据记录索引获取这些块。当新的 token 产生时，就会进行新的区块分配。这些块的大小是固定的，消除了因不同请求需要不同分配等挑战而产生的低效率。这极大地限制了内存浪费，从而实现了更大的批量大小（从而提高了吞吐量）。常用的 vLLM 推理框架就是采用的 PagedAttention 技术。

5. 模型优化技术

到目前为止，我们已经讨论了 LLM 消耗内存的不同方式、跨多个不同 GPU 分配内存的一些方式，以及优化注意力机制和 KV 缓存。还有多种模型优化技术可以通过修改模型权重本身来减少每个 GPU 上的内存使用。GPU 还具有专用硬件来加速这些修改值的运算，从而为模型提供更多加速。

5.1 量化⁺ (Quantization)

量化是降低模型权重和激活精度的过程。大多数模型都以 32 或 16 位精度进行训练，其中每个参数和激活元素占用 32 或 16 位内存（单精度浮点）。然而，大多数深度学习模型可以用每个值八个甚至更少的位来有效表示。



上图显示了一种可能的量化方法之前和之后的值分布。在这种情况下，会丢失一些精度，并且剪裁会丢失一些动态范围，从而允许以更小的格式表示值。

但是降低模型的精度可以带来多种好处。如果模型占用的内存空间较少，则可以在相同数量的硬件上安运行更大的模型。量化还意味着可以在相同的带宽上传输更多参数，这有助于加速带宽有限的模型。

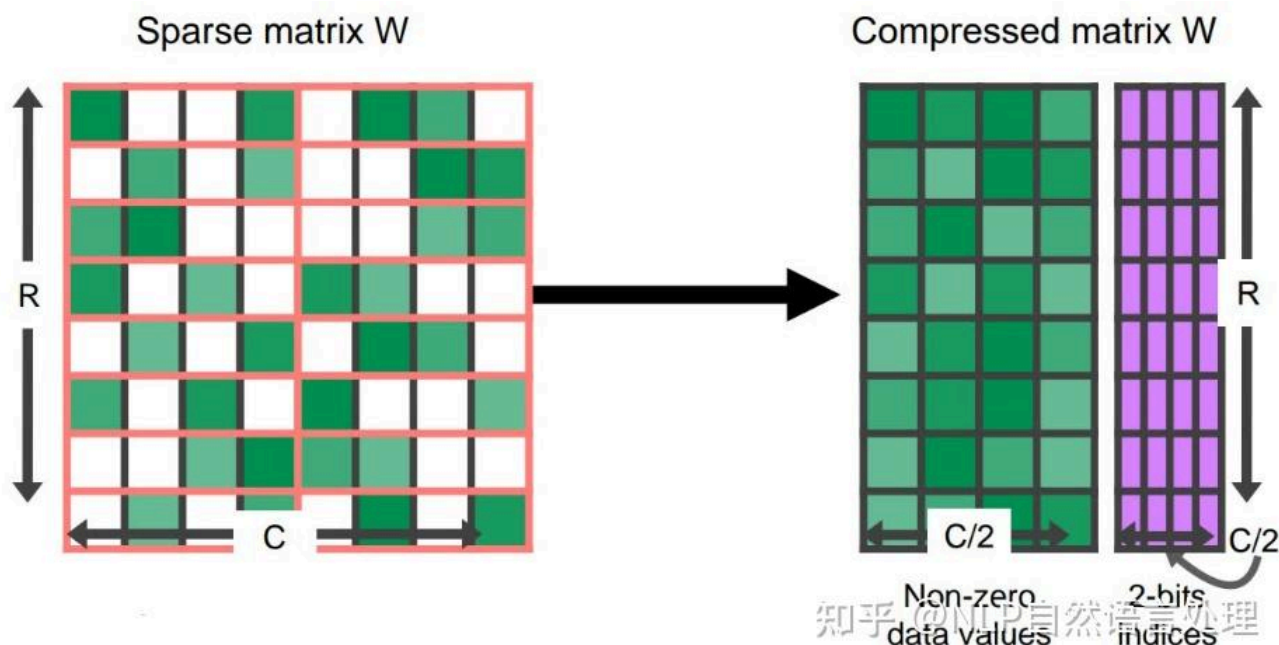
LLM 有许多不同的量化技术，涉及降低激活、权重或两者的精度。量化权重要简单得多，因为它们在训练后是固定的。然而，这可能会留下一些性能问题，因为激活仍然保持在更高的精度。GPU 没有用于乘以 INT8 和 FP16 数字的专用硬件，因此必须将权重转换回更高精度以进行实际运算。

还可以量化激活、Transformer 块和网络层的输入，但这也有其自身的挑战。激活向量通常包含异常值，有效地增加了它们的动态范围，并使以比权重更低的精度表示这些值变得更具挑战性。

一种选择是通过模型传递代表性数据集并选择以比其他激活更高的精度表示某些激活来找出这些异常值可能出现的位置 (LLM.int8())。另一种选择是借用易于量化的权重的动态范围，并在激活中重用该范围。

5.2 稀疏* (Sparsity)

与量化类似，事实证明，许多深度学习模型对于修剪或用 0 本身替换某些接近 0 的值具有鲁棒性。稀疏矩阵是许多元素为 0 的矩阵。这些矩阵可以用压缩形式表示，比完整的稠密矩阵占用的空间更少。



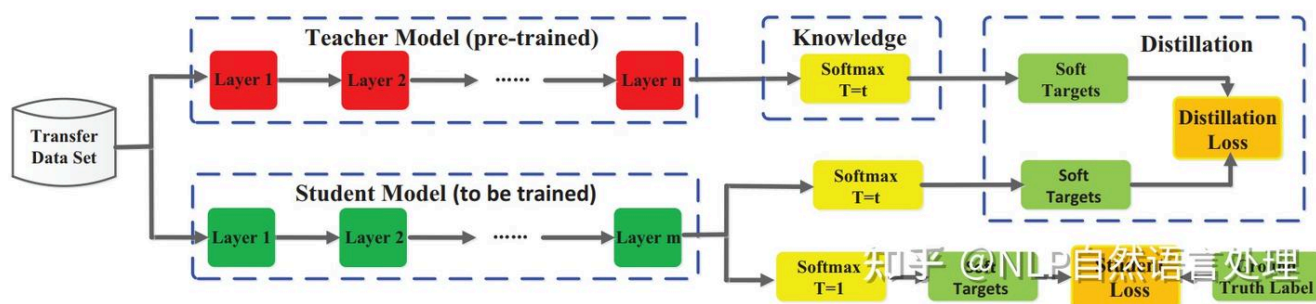
GPU 尤其具有针对某种结构化稀疏性的硬件加速，其中每四个值中有两个由零表示。稀疏表示还可以与量化相结合，以实现更大的执行速度。寻找以稀疏格式表示大型语言模型的最佳方法仍然是一个活跃的研究领域，并为未来提高推理速度提供了一个有希望的方向。

5.3 蒸馏⁺ (Distillation)

缩小模型大小的另一种方法是通过称为蒸馏的过程将其知识转移到较小的模型。此过程涉及训练较小的模型（称为学生）来模仿较大模型（教师）的行为。蒸馏模型的成功例子包括 DistilBERT，它将 BERT 模型压缩了 40%，同时保留了 97% 的语言理解能力，速度提高了 60%。

虽然 LLMs 中的蒸馏是一个活跃的研究领域，但神经网络的一般方法首次在论文 *Distilling the Knowledge in a Neural Network* (arxiv.org/abs/1503.02531) 中提出：学生网络经过训练，可以反映较大教师网络的性能，使用损失函数来测量其输出之间的差异。该目标还可能包括将学生的输出与真实标签进行匹配的原始损失函数。

匹配的教师输出可以是最后一层（称为 logits）或中间层激活。



上图显示了知识蒸馏通用的总体框架 (arxiv.org/pdf/2006.05521)。教师的 logits 是学生使用蒸馏损失进行优化的软目标。其他蒸馏方法可能会使用其他损失措施来从老师那里“蒸馏”知识。

蒸馏的另一种方法是使用教师合成的数据对 LLMs 学生进行监督培训，这在人工注释稀缺或不可用时特别有用。一步一步蒸馏！更进一步，除了作为基本事实的标签之外，还从 LLMs 教师那里提取基本原理。这些基本原理作为中间推理步骤，以数据有效的方式培训规模较小的 LLMs。

值得注意的是，当今许多最先进的 LLMs 都有限制性许可证，禁止使用他们的成果来训练其他 LLMs，这使得找到合适的教师模型具有挑战性。

6. 模型服务技术

模型执行通常受内存带宽限制，特别是权重中的带宽限制。即使在应用了前面描述的所有模型优化之后，它仍然很可能受到内存限制。因此，在加载模型权重时尽可能多地处理它们。换句话说，尝试并行。可以采取两种方法：

- 动态批处理(In-flight batching)：同时执行多个不同的请求。
- 预测推理(Speculative inference)：并行执行序列的多个不同步骤以尝试节省时间。

6.1 动态批处理 (In-flight batching)

LLMs 具有一些独特的执行特征，这些特征可能导致在实践中难以有效地处理批量请求。一个模型可以同时用于多种不同的任务。从聊天机器人中的简单问答响应到文档摘要或代码块的生成，工作负载是高度动态的，输出大小变化几个数量级。

这种多功能性使得批处理请求并有效地并行执行它们变得具有挑战性，这是服务神经网络的常见优化。这可能会导致某些请求比其他请求更早完成。

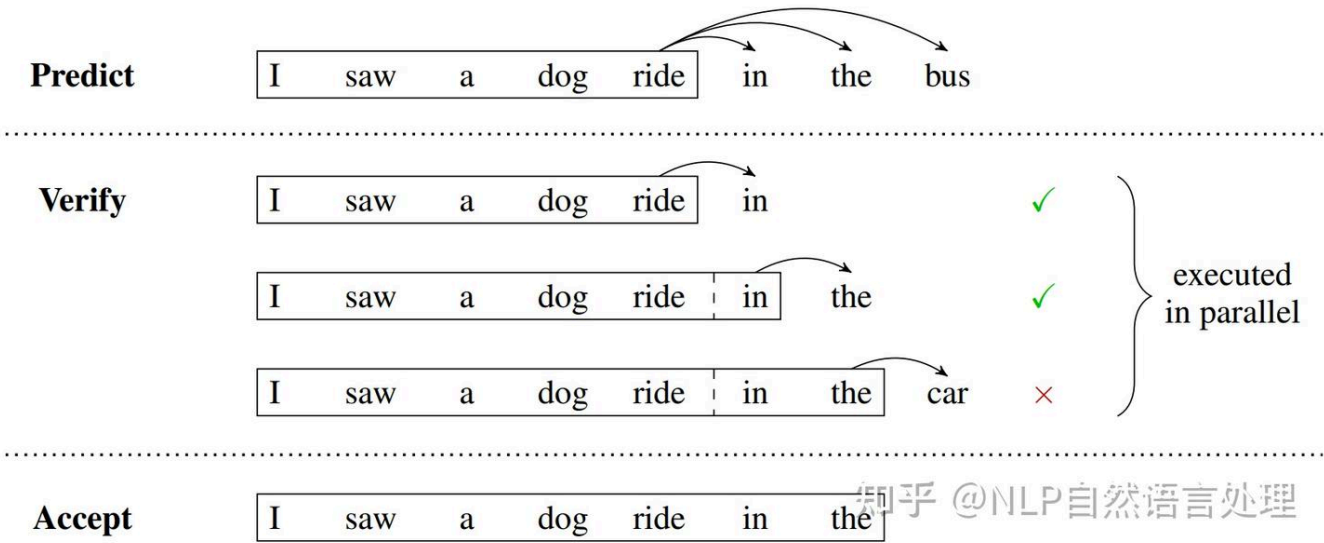
为了管理这些动态负载，许多 LLMs 服务解决方案包括一种称为连续或动态批处理的优化调度技术。这利用了这样一个事实：LLMs 的整个文本生成过程可以分解为模型上的多次执行迭代。

通过动态批处理，服务器运行时会立即从批处理中剔除已完成的序列，而不是等待整个批处理完成后再继续处理下一组请求。然后，它开始执行新请求，而其他请求仍在进行中。因此，动态批处理可以极大地提高实际用例中 GPU 的整体利用率。

6.2 预测推理 (Speculative inference)

预测推理也称为推测采样、辅助生成或分块并行解码，是并行执行 LLM 的另一种方式。通常，GPT 风格的大语言模型是自回归模型，逐个生成文本标记。

生成的每个标记都依赖于它之前的所有标记来提供上下文。这意味着在常规执行中，不可能从同一个序列并行生成多个 token，必须等待第 n 个 token 生成后才能生成 n+1 个 token。



上图为预测推理的示例，其中临时模型临时预测并行验证或拒绝的多个未来步骤。在这种情况下，临时模型中的前两个预测 token 被接受，而最后一个在继续生成之前被拒绝并删除。

预测性抽样提供了一种解决方法。这种方法的基本思想是使用一些“更便宜”的过程来生成几个 token 长的临时序列。然后，并行执行多个步骤的主要“验证”模型，使用廉价临时序列作为需要的执行步骤的“预测”上下文。

如果验证模型生成与临时序列相同的 token，那么就接受这些 token 作为输出。否则，可以丢弃第一个不匹配标记之后的所有内容，并使用新的临时序列重复该过程。

如何生成临时 token 有许多不同的选项，每个选项都有不同的权衡。可以训练多个模型，或在单个预训练模型上微调多个头，以预测未来多个步骤的标记。或者，可以使用小型模型作为临时模型，使用更大、功能更强大的模型作为验证器。

[1]Transformer|前馈神经网络 (FFN)

[2]Transformer|从 MHA 到 DeepSeek MLA!

[3]Transformer|注意力机制 Attention

[4]Transformer|MoE 架构 (含 DeepSeek)

[5]Transformer|归一化(Normalization)

[6]Transformer|位置编码 (DeepSeek 位置编码)

发布于 2025-06-20 14:56 · 上海

LLM 推理

▲ 赞同 285 ▼ ● 6 条评论 ↗ 分享 ❤ 喜欢 ★ 收藏 📄 申请转载 ...